

SelVerIR: Self-Verifying Programming Language
Intermediate Representation Syntax and Semantics Document
v0.2

Yusuf Demir
Yağız Kaan Aydoğdu

March 10, 2026

1 Syntax

$$\begin{aligned}\langle IR \rangle &::= \{ \langle label \rangle \langle Command \rangle \} \\ \langle Command \rangle &::= \text{DECL } \langle type \rangle , \langle ident \rangle \\ &\quad | \text{PUSH } \langle imm \rangle \\ &\quad | \text{iSTORE } \langle ident \rangle \mid \text{fSTORE } \langle ident \rangle \\ &\quad | \text{LOAD } \langle ident \rangle \\ &\quad | \text{ADD} \\ &\quad | \text{SUB} \\ &\quad | \text{MUL} \\ &\quad | \text{iDIV} \mid \text{fDIV} \\ &\quad | \text{LEN} \\ &\quad | \text{EQ} \\ &\quad | \text{LT} \\ &\quad | \text{LE} \\ &\quad | \text{GT} \\ &\quad | \text{GE} \\ &\quad | \text{NEG} \\ &\quad | \text{AND} \\ &\quad | \text{OR} \\ &\quad | \text{XOR} \\ &\quad | \text{JZ } \langle int \rangle \\ &\quad | \text{GOTO } \langle int \rangle \\ &\quad | \text{NOOP} \\ \\ \langle type \rangle &::= \text{INT} \mid \text{FLOAT} \mid \text{LIST}[\langle type \rangle , \langle int \rangle] \\ \langle ident \rangle &::= [\text{A-Za-z}][\text{A-Za-z0-9}]^* \\ \langle label \rangle &::= [0-9]^+ : \\ \langle int \rangle &::= [0-9]^+ \\ \langle float \rangle &::= [0-9]^+ . [0-9]^+ \\ \langle imm \rangle &::= \langle int \rangle \mid \langle float \rangle\end{aligned}$$

2 Semantics

2.1 Mathematical notions

The **SelVeri** intermediate representation, **SelVerIR** is executed on a *stack-based* abstract machine (**AM**) and aims to be provably correct implementation of the high-level **SelVeri** language.

2.1.1 Stack

At any moment of the execution of a **SelVerIR** program, its stack, denoted as e , is a an ordered finite sequence consisting elements are from **Val**, which will be used in the flow of the program in the *LIFO* order. As expected, e supports **PUSH** and (indirectly) **POP** operations. For convenience, the empty stack is denoted separately, as ϵ .

2.1.2 Program counter

Intuitively, one can think of any **SelVerIR** program, P as an ordered list of commands. Program counter, pc , represents the index of the command that is to be executed next. Control-flow statements such as **GOTO** and **JZ** can alter the program counter in a non-sequential manner.

2.1.3 IR-configuration

An IR-configuration is the representation of the execution state in the form of a 5-tuple $\langle c, e, \sigma, s, pc \rangle$ where

- c is the list of commands representing the remaining part of the program **SelVerIR** program, i.e, $c = P[pc:]$.
- e is the stack of the execution state,
- σ is the program state,
- s is the scope,
- pc is the program counter.

2.2 Small-step operational semantics of SelVerIR commands

Let P be any SelVerIR program throughout the section 2.2.

2.2.1 Small-step operational semantics of state-related commands

$\langle \text{DECL } t : c, e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, e, \sigma[x \mapsto \mathbf{0}], s[x \mapsto t], pc + 1 \rangle \\ \text{if } x \in \mathbf{Var} \wedge t \in \mathbf{Type} \wedge s(x) = \mathbf{Void} \\ \langle c, e, \hat{\sigma}, s, pc + 1 \rangle & \text{otherwise} \end{cases}$
$\langle \text{iSTORE } x : c, z : e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, e, \sigma[x \mapsto z], s, pc + 1 \rangle & \text{if } s(x) = \mathbf{Int} \wedge z \in \mathbb{Z} \\ \langle c, e, \hat{\sigma}, s, pc + 1 \rangle & \text{otherwise} \end{cases}$
$\langle \text{fSTORE } x : c, r : e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, e, \sigma[x \mapsto r], s, pc + 1 \rangle & \text{if } s(x) = \mathbf{Float} \wedge r \in \mathbb{R} \\ \langle c, e, \hat{\sigma}, s, pc + 1 \rangle & \text{otherwise} \end{cases}$
$\langle \text{LOAD } x : c, e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, \sigma(x, s) : e, \sigma, s, pc + 1 \rangle & \text{if } x \in \mathbf{Var} \wedge \sigma(x, s) \neq \perp \\ \langle c, e, \sigma, s, pc + 1 \rangle & \text{otherwise} \end{cases}$
$\langle \text{PUSH } v : c, e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, \mathcal{V}(v) : e, \sigma, s, pc + 1 \rangle \text{ if } v \in \mathbf{Imm}$

Table 1: Operational semantics of state-related commands

2.2.2 Small-step operational semantics of arithmetic commands

$\langle \text{ADD } : c, v_1 : v_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, (v_1 + v_2) : e, \sigma, s, pc + 1 \rangle \text{ if } v_1, v_2 \in \mathbb{R}$
$\langle \text{SUB } : c, v_1 : v_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, (v_1 - v_2) : e, \sigma, s, pc + 1 \rangle \text{ if } v_1, v_2 \in \mathbb{R}$
$\langle \text{MUL } : c, v_1 : v_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, (v_1 \cdot v_2) : e, \sigma, s, pc + 1 \rangle \text{ if } v_1, v_2 \in \mathbb{R}$
$\langle \text{iDIV } : c, z_1 : z_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, [z_1/z_2] : e, \sigma, s, pc + 1 \rangle & \text{if } z_1, z_2 \in \mathbb{Z} \wedge z_2 \neq 0 \\ \langle c, \perp : e, \hat{\sigma}, s, pc + 1 \rangle & \text{otherwise} \end{cases}$
$\langle \text{fDIV } : c, r_1 : r_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, r_1/r_2 : e, \sigma, s, pc + 1 \rangle & \text{if } r_1, r_2 \in \mathbb{R} \wedge r_2 \neq 0 \\ \langle c, \perp : e, \hat{\sigma}, s, pc + 1 \rangle & \text{otherwise} \end{cases}$
$\langle \text{LEN } x : c, e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, 0 : e, \sigma, s, pc + 1 \rangle & \text{if } s(x) \in \{\mathbf{Int}, \mathbf{Float}\} \\ \langle c, n : e, \sigma, s, pc + 1 \rangle & \text{if } \exists t \in \mathbf{Type} \cdot \exists n \in \mathbb{N}^+ \cdot s(x) = \mathbf{List}[t, n] \\ \langle c, \perp : e, \hat{\sigma}, s, pc + 1 \rangle & \text{otherwise} \end{cases}$

Table 2: Operational semantics of arithmetic commands

As one can notice from the table above, we do not have type-specific commands for ADD, SUB and MUL as these operations are closed under $\mathbb{Z}$ ¹. However, since this is not the case for division, we have separate instructions for integer and float division, namely iDIV and fDIV. A similar approach will be followed for the boolean expressions in the next section.

¹Although syntactically different in SelVeri high level language; semantically, we do not interpret any integer differently than its real number counterpart.

2.2.3 Small-step operational semantics of boolean commands

$\langle \text{EQ} : c, v_1 : v_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, (v_1 = v_2) : e, \sigma, s, pc + 1 \rangle$ if $v_1, v_2 \in \mathbb{R}$
$\langle \text{LE} : c, v_1 : v_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, (v_1 \leq v_2) : e, \sigma, s, pc + 1 \rangle$ if $v_1, v_2 \in \mathbb{R}$
$\langle \text{LT} : c, v_1 : v_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, (v_1 < v_2) : e, \sigma, s, pc + 1 \rangle$ if $v_1, v_2 \in \mathbb{R}$
$\langle \text{GE} : c, v_1 : v_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, (v_1 \geq v_2) : e, \sigma, s, pc + 1 \rangle$ if $v_1, v_2 \in \mathbb{R}$
$\langle \text{GT} : c, v_1 : v_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, (v_1 > v_2) : e, \sigma, s, pc + 1 \rangle$ if $v_1, v_2 \in \mathbb{R}$
$\langle \text{AND} : c, b_1 : b_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, 1 : e, \sigma, s, pc + 1 \rangle & \text{if } b_1 = 1 \wedge b_2 = 1 \\ \langle c, 0 : e, \sigma, s, pc + 1 \rangle & \text{if } b_1 = 0 \vee b_2 = 0 \end{cases}$
$\langle \text{OR} : c, b_1 : b_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, 1 : e, \sigma, s, pc + 1 \rangle & \text{if } b_1 = 1 \vee b_2 = 1 \\ \langle c, 0 : e, \sigma, s, pc + 1 \rangle & \text{if } b_1 = 0 \wedge b_2 = 0 \end{cases}$
$\langle \text{XOR} : c, b_1 : b_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, 1 : e, \sigma, s, pc + 1 \rangle & \text{if } b_1 \neq b_2 \\ \langle c, 0 : e, \sigma, s, pc + 1 \rangle & \text{if } b_1 = b_2 \end{cases}$
$\langle \text{NEG} : c, b : e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, 1 : e, \sigma, s, pc + 1 \rangle & \text{if } b = 0 \\ \langle c, 0 : e, \sigma, s, pc + 1 \rangle & \text{if } b = 1 \end{cases}$

Table 3: Operational semantics of boolean commands

2.2.4 Small-step operational semantics of control-flow commands

$\langle \text{JZ } z : c, b : e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, e, \sigma, s, pc + 1 \rangle & \text{if } b = 1 \\ \langle P[\mathcal{I}(z) :], e, \sigma, s, \mathcal{I}(z) \rangle & \text{if } b = 0 \wedge z \in \mathbf{Imm} \cap \mathbf{Int} \end{cases}$
$\langle \text{GOTO } z : c, e, \sigma, s, pc \rangle$	$\triangleright$	$\langle P[\mathcal{I}(z) :], e, \sigma, s, \mathcal{I}(z) \rangle$ if $z \in \mathbf{Imm} \cap \mathbf{Int}$
$\langle \text{NOOP} : c, e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, e, \sigma, s, pc + 1 \rangle$
$\langle i : c, e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, e, \hat{\sigma}, s, pc + 1 \rangle$ where i is any command

Table 4: Operational semantics of control-flow commands

The reader should note that any configuration which is not explicitly stated in the tables above lead to an error state, $\hat{\sigma}$ without any other change to the original state σ .

3 Compilation

Let **Code** and **CodeIR** be the sets of all **SelVeri** and **SelVerIR** programs, respectively. Using these sets, we will inductively define a compiler function:

$$\mathcal{C} : \mathbf{Code} \rightarrow \mathbf{CodeIR}$$

3.1 Compilation of arithmetic expressions

In order to formally define the compilation of arithmetic expressions, we define a helper function:

$$\mathcal{CA} : \mathbf{AExp} \rightarrow \mathbf{CodeIR}$$

$$\begin{aligned} \mathcal{CA}(x) &= \text{PUSH } x && \text{if } x \in \mathbf{Imm} \cap (\mathbf{Int} \cup \mathbf{Float}) \\ \mathcal{CA}(x) &= \text{LOAD } x && \text{if } x \in \mathbf{Var} \\ \mathcal{CA}(\text{len}(x)) &= \text{LEN } x \\ \mathcal{CA}(a_1 + a_2) &= \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \text{ADD} \\ \mathcal{CA}(a_1 * a_2) &= \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \text{MUL} \\ \mathcal{CA}(a_1 - a_2) &= \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \text{SUB} \\ \mathcal{CA}(a_1 / a_2) &= \begin{cases} \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \text{iDIV} & \text{if } (a_1/a_2) \text{ has no subterm of type}^2 \\ & \mathbf{Float} \\ \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \text{fDIV} & \text{if } (a_1/a_2) \text{ has at least one subterm} \\ & \text{of type } \mathbf{Float} \end{cases} \end{aligned}$$

²The types of subterms are determined by the compiler during the compile time.

3.2 Compilation of boolean expressions

In order to formally define the compilation of boolean expressions, we define a helper function:

$$\mathcal{CB} : \mathbf{BExp} \rightarrow \mathbf{CodeIR}$$

$$\mathcal{CB}(\mathbf{true}) = \mathbf{PUSH\ 1}$$

$$\mathcal{CB}(\mathbf{false}) = \mathbf{PUSH\ 0}$$

$$\mathcal{CB}(a) = \mathcal{CA}(a) : \mathbf{PUSH\ 0} : \mathbf{EQ} : \mathbf{NEG} \quad \text{if } a \in \mathbf{AExp}$$

$$\mathcal{CB}(a_1 = a_2) = \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \mathbf{EQ}$$

$$\mathcal{CB}(a_1 \leq a_2) = \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \mathbf{LE}$$

$$\mathcal{CB}(\mathbf{not } b) = \mathcal{CB}(b) : \mathbf{NEG}$$

$$\mathcal{CB}(b_1 \mathbf{ and } b_2) = \mathcal{CB}(b_2) : \mathcal{CB}(b_1) : \mathbf{AND}$$

3.3 Compilation of statements

Finally, with the help of $\mathcal{CA}$ and $\mathcal{CB}$, we are ready to properly define the compilation function:

$$\mathcal{C} : \mathbf{Code} \rightarrow \mathbf{CodeIR}$$

$$\mathcal{C}(x : t) = \text{DECL } t$$

$$\mathcal{C}(x := a) = \begin{cases} \mathcal{CA}(a) : \text{iSTORE } x & \text{if type}^4 \text{ of } x \text{ is } \mathbf{Int}. \\ \mathcal{CA}(a) : \text{fSTORE } x & \text{if type of } x \text{ is } \mathbf{Float}. \end{cases}$$

$$\mathcal{C}(\text{pass}) = \text{NOOP}$$

$$\mathcal{C}(S_1; S_2) = \mathcal{C}(S_1) : \mathcal{C}(S_2)$$

$$\mathcal{C}(\text{if } b \text{ then } S \text{ fi}) = \mathcal{CB}(b) : \text{JZ } pc_{\text{NOOP}} : \mathcal{C}(S) : \text{NOOP}$$

$$\mathcal{C}(\text{if } b \text{ then } S_1 \text{ else } S_2 \text{ fi}) = \mathcal{CB}(b) : \text{JZ } pc_{S_2} : \mathcal{C}(S_1) : \text{GOTO } pc_{\text{NOOP}} : \mathcal{C}(S_2) : \text{NOOP}$$

$$\mathcal{C}(\text{while } b \text{ do } S \text{ od}) = \mathcal{CB}(b) : \text{JZ } pc_{\text{NOOP}} : \mathcal{C}(S) : \text{GOTO } pc_b : \text{NOOP}$$

Please note that, during the compilation phase, compiler labels each compiled command with a corresponding program counter, using a simple incrementing counter. Therefore, pc variables seen in the compilation of the control-flow statements are actually replaced with syntactic integer-immediates.

⁶The type of variables are determined by the compiler during the compile time, with the help of scope.

4 Provably Correct Implementation

Now that the compilation function is explicitly defined, we need to prove the correctness of our implementation; by showing that given any **SelVeri** program P , we have

$$\langle P, \tilde{\sigma}, \tilde{s} \rangle \rightarrow^* \underline{\sigma}_f \implies \langle \mathcal{C}(P), \epsilon, \tilde{\sigma}, \tilde{s}, 0 \rangle \triangleright^* \underline{\sigma}_f$$

4.1 Correctness of expressions

4.1.1 Arithmetic

Given any program stack e , non-erroneous program state σ , scope s , program counter pc , we have:

$$\forall a \in \mathbf{AExp} \cdot \langle \mathcal{CA}(a) : c, e, \sigma, s, pc \rangle \triangleright^* \langle c, \mathcal{A}(a, \sigma, s) : e, \underline{\sigma}, s, pc + l \rangle$$

where l is the number of commands in $\mathcal{CA}(a)$.

Proof. We will prove this by structural induction on arithmetic expressions.

- **case 1:** $a = v$ for some $v \in \mathbf{Imm} \cap (\mathbf{Int} \cup \mathbf{Float})$

$$\begin{aligned} \langle \mathcal{CA}(a) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CA}(v) : c, e, \sigma, s, pc \rangle \\ &= \langle \mathbf{PUSH} \ v : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CA} \text{)} \\ &\triangleright \langle c, \mathcal{V}(v) : e, \sigma, s, pc + 1 \rangle && \text{(operational semantics)} \\ &= \langle c, \mathcal{A}(v, \sigma, s) : e, \sigma, s, pc + 1 \rangle && \text{(definition of } \mathcal{A} \text{)} \\ &= \langle c, \mathcal{A}(a, \sigma, s) : e, \sigma, s, pc + 1 \rangle \end{aligned}$$

- **case 2:** $a = x$ for some $x \in \mathbf{Var}$

$$\begin{aligned} \langle \mathcal{CA}(a) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CA}(x) : c, e, \sigma, s, pc \rangle \\ &= \langle \mathbf{LOAD} \ x : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CA} \text{)} \\ &\triangleright \langle c, \sigma(x, s) : e, \sigma, s, pc + 1 \rangle && \text{(operational semantics)} \\ &= \langle c, \mathcal{A}(x, \sigma, s) : e, \sigma, s, pc + 1 \rangle && \text{(definition of } \mathcal{A} \text{)} \\ &= \langle c, \mathcal{A}(a, \sigma, s) : e, \sigma, s, pc + 1 \rangle \end{aligned}$$

- **case 3:** $a = \mathbf{len}(x)$ for some $x \in \mathbf{Var}$

- **subcase 3.1:** $s(x) \in \{\mathbf{Int}, \mathbf{Float}\}$

$$\begin{aligned} \langle \mathcal{CA}(a) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CA}(\mathbf{len}(x)) : c, e, \sigma, s, pc \rangle \\ &= \langle \mathbf{LEN} \ x : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CA} \text{)} \\ &\triangleright \langle c, 0 : e, \sigma, s, pc + 1 \rangle && \text{(operational semantics)} \\ &= \langle c, \mathcal{A}(\mathbf{len}(x), \sigma, s) : e, \sigma, s, pc + 1 \rangle && \text{(definition of } \mathcal{A} \text{)} \\ &= \langle c, \mathcal{A}(a, \sigma, s) : e, \sigma, s, pc + 1 \rangle \end{aligned}$$

- **subcase 3.2:** $\exists t \in \mathbf{Type} \cdot \exists n \in \mathbb{N}^+ \cdot s(x) = \mathbf{List}[t, n]$

$$\begin{aligned} \langle \mathcal{CA}(a) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CA}(\mathbf{len}(x)) : c, e, \sigma, s, pc \rangle \\ &= \langle \mathbf{LEN} \ x : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CA} \text{)} \\ &\triangleright \langle c, n : e, \sigma, s, pc + 1 \rangle && \text{(operational semantics)} \\ &= \langle c, \mathcal{A}(\mathbf{len}(x), \sigma, s) : e, \sigma, s, pc + 1 \rangle && \text{(definition of } \mathcal{A} \text{)} \\ &= \langle c, \mathcal{A}(a, \sigma, s) : e, \sigma, s, pc + 1 \rangle \end{aligned}$$

– **subcase 3.3:** $s(x) \notin \{\mathbf{Int}, \mathbf{Float}\} \wedge \forall t \in \forall Type \cdot \forall n \in \mathbb{N}^+ \cdot s(x) \neq \mathbf{List}[t, n]$

$$\begin{aligned}
\langle \mathcal{CA}(a) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CA}(\mathbf{len}(x)) : c, e, \sigma, s, pc \rangle \\
&= \langle \mathbf{LEN} x : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CA} \text{)} \\
&\triangleright \langle c, \perp : e, \hat{\sigma}, s, pc + 1 \rangle && \text{(operational semantics)} \\
&= \langle c, \mathcal{A}(\mathbf{len}(x), \sigma, s) : e, \hat{\sigma}, s, pc + 1 \rangle && \text{(definition of } \mathcal{A} \text{)} \\
&= \langle c, \mathcal{A}(a, \sigma, s) : e, \hat{\sigma}, s, pc + 1 \rangle
\end{aligned}$$

• **case 4:** $a = a_1 + a_2$ for some $a_1, a_2 \in \mathbf{AExp}$

Assume for an induction on the correctness of the compilation of a_1 and a_2 .

$$\begin{aligned}
\langle \mathcal{CA}(a) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CA}(a_1 + a_2) : c, e, \sigma, s, pc \rangle \\
&= \langle \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \mathbf{ADD} : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CA} \text{)} \\
&\triangleright^* \langle \mathcal{CA}(a_1) : \mathbf{ADD} : c, \mathcal{A}(a_2, \sigma, s) : e, \sigma, s, pc + l_2 \rangle && \text{(induction hypothesis 2)} \\
&\triangleright^* \langle \mathbf{ADD} : c, \mathcal{A}(a_1, \sigma, s) : \mathcal{A}(a_2, \sigma, s) : e, \sigma, s, pc + l_2 + l_1 \rangle && \text{(induction hypothesis 1)} \\
&\triangleright \langle c, (\mathcal{A}(a_1, \sigma, s) + \mathcal{A}(a_2, \sigma, s)) : e, \sigma, s, pc + l_2 + l_1 + 1 \rangle && \text{(operational semantics)} \\
&= \langle c, \mathcal{A}(a_1 + a_2, \sigma, s) : e, \sigma, s, pc + l_2 + l_1 + 1 \rangle && \text{(definition of } \mathcal{A} \text{)} \\
&= \langle c, \mathcal{A}(a, \sigma, s) : e, \sigma, s, pc + l_2 + l_1 + 1 \rangle
\end{aligned}$$

• **case 5:** $a = a_1 * a_2$ for some $a_1, a_2 \in \mathbf{AExp}$

Assume for an induction on the correctness of the compilation of a_1 and a_2 .

$$\begin{aligned}
\langle \mathcal{CA}(a) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CA}(a_1 * a_2) : c, e, \sigma, s, pc \rangle \\
&= \langle \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \mathbf{MUL} : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CA} \text{)} \\
&\triangleright^* \langle \mathcal{CA}(a_1) : \mathbf{MUL} : c, \mathcal{A}(a_2, \sigma, s) : e, \sigma, s, pc + l_2 \rangle && \text{(induction hypothesis 2)} \\
&\triangleright^* \langle \mathbf{MUL} : c, \mathcal{A}(a_1, \sigma, s) : \mathcal{A}(a_2, \sigma, s) : e, \sigma, s, pc + l_2 + l_1 \rangle && \text{(induction hypothesis 1)} \\
&\triangleright \langle c, (\mathcal{A}(a_1, \sigma, s) \cdot \mathcal{A}(a_2, \sigma, s)) : e, \sigma, s, pc + l_2 + l_1 + 1 \rangle && \text{(operational semantics)} \\
&= \langle c, \mathcal{A}(a_1 * a_2, \sigma, s) : e, \sigma, s, pc + l_2 + l_1 + 1 \rangle && \text{(definition of } \mathcal{A} \text{)} \\
&= \langle c, \mathcal{A}(a, \sigma, s) : e, \sigma, s, pc + l_2 + l_1 + 1 \rangle
\end{aligned}$$

• **case 6:** $a = a_1 - a_2$ for some $a_1, a_2 \in \mathbf{AExp}$

Assume for an induction on the correctness of the compilation of a_1 and a_2 .

$$\begin{aligned}
\langle \mathcal{CA}(a) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CA}(a_1 - a_2) : c, e, \sigma, s, pc \rangle \\
&= \langle \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \mathbf{SUB} : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CA} \text{)} \\
&\triangleright^* \langle \mathcal{CA}(a_1) : \mathbf{SUB} : c, \mathcal{A}(a_2, \sigma, s) : e, \sigma, s, pc + l_2 \rangle && \text{(induction hypothesis 2)} \\
&\triangleright^* \langle \mathbf{SUB} : c, \mathcal{A}(a_1, \sigma, s) : \mathcal{A}(a_2, \sigma, s) : e, \sigma, s, pc + l_2 + l_1 \rangle && \text{(induction hypothesis 1)} \\
&\triangleright \langle c, (\mathcal{A}(a_1, \sigma, s) - \mathcal{A}(a_2, \sigma, s)) : e, \sigma, s, pc + l_2 + l_1 + 1 \rangle && \text{(operational semantics)} \\
&= \langle c, \mathcal{A}(a_1 - a_2, \sigma, s) : e, \sigma, s, pc + l_2 + l_1 + 1 \rangle && \text{(definition of } \mathcal{A} \text{)} \\
&= \langle c, \mathcal{A}(a, \sigma, s) : e, \sigma, s, pc + l_2 + l_1 + 1 \rangle
\end{aligned}$$

• **case 7.1:** $a = a_1 / a_2$ for some $a_1, a_2 \in \mathbf{AExp}$ and (a_1 / a_2) has no subterm of type **Float**

Assume for an induction on the correctness of the compilation of a_1 and a_2 .

– **subcase 7.1.1:** $\mathcal{A}(a_2, \sigma, s) \neq 0$

$$\begin{aligned}
\langle \mathcal{CA}(a) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CA}(a_1/a_2) : c, e, \sigma, s, pc \rangle \\
&= \langle \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \text{iDIV} : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CA} \text{)} \\
&\triangleright^* \langle \mathcal{CA}(a_1) : \text{iDIV} : c, \mathcal{A}(a_2, \sigma, s) : e, \sigma, s, pc + l_2 \rangle && \text{(induction hypothesis 2)} \\
&\triangleright^* \langle \text{iDIV} : c, \mathcal{A}(a_1, \sigma, s) : \mathcal{A}(a_2, \sigma, s) : e, \sigma, s, pc + l_2 + l_1 \rangle && \text{(induction hypothesis 1)} \\
&\triangleright \langle c, \lfloor \mathcal{A}(a_1, \sigma, s) / \mathcal{A}(a_2, \sigma, s) \rfloor : e, \sigma, s, pc + l_2 + l_1 + 1 \rangle && \text{(operational semantics)} \\
&= \langle c, \mathcal{A}(a_1/a_2, \sigma, s) : e, \sigma, s, pc + l_2 + l_1 + 1 \rangle && \text{(definition of } \mathcal{A} \text{)} \\
&= \langle c, \mathcal{A}(a, \sigma, s) : e, \sigma, s, pc + l_2 + l_1 + 1 \rangle
\end{aligned}$$

– **subcase 7.1.2:** $\mathcal{A}(a_2, \sigma, s) = 0$

$$\begin{aligned}
\langle \mathcal{CA}(a) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CA}(a_1/a_2) : c, e, \sigma, s, pc \rangle \\
&= \langle \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \text{iDIV} : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CA} \text{)} \\
&\triangleright^* \langle \mathcal{CA}(a_1) : \text{iDIV} : c, \mathcal{A}(a_2, \sigma, s) : e, \sigma, s, pc + l_2 \rangle && \text{(induction hypothesis 2)} \\
&= \langle \mathcal{CA}(a_1) : \text{iDIV} : c, 0 : e, \sigma, s, pc + l_2 \rangle \\
&\triangleright^* \langle \text{iDIV} : c, \mathcal{A}(a_1, \sigma, s) : 0 : e, \sigma, s, pc + l_2 + l_1 \rangle && \text{(induction hypothesis 1)} \\
&\triangleright \langle c, \perp : e, \hat{\sigma}, s, pc + l_2 + l_1 + 1 \rangle && \text{(operational semantics)} \\
&= \langle c, \mathcal{A}(a_1/a_2, \sigma, s) : e, \hat{\sigma}, s, pc + l_2 + l_1 + 1 \rangle && \text{(definition of } \mathcal{A} \text{)} \\
&= \langle c, \mathcal{A}(a, \sigma, s) : e, \hat{\sigma}, s, pc + l_2 + l_1 + 1 \rangle
\end{aligned}$$

• **case 7.2:** $a = a_1/a_2$ for some $a_1, a_2 \in \mathbf{AExp}$ and (a_1/a_2) has at least one subterm of type **Float**

Assume for an induction on the correctness of the compilation of a_1 and a_2 .

– **subcase 7.2.1:** $\mathcal{A}(a_2, \sigma, s) \neq 0.0$

$$\begin{aligned}
\langle \mathcal{CA}(a) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CA}(a_1/a_2) : c, e, \sigma, s, pc \rangle \\
&= \langle \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \text{fDIV} : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CA} \text{)} \\
&\triangleright^* \langle \mathcal{CA}(a_1) : \text{fDIV} : c, \mathcal{A}(a_2, \sigma, s) : e, \sigma, s, pc + l_2 \rangle && \text{(induction hypothesis 2)} \\
&\triangleright^* \langle \text{fDIV} : c, \mathcal{A}(a_1, \sigma, s) : \mathcal{A}(a_2, \sigma, s) : e, \sigma, s, pc + l_2 + l_1 \rangle && \text{(induction hypothesis 1)} \\
&\triangleright \langle c, \mathcal{A}(a_1, \sigma, s) / \mathcal{A}(a_2, \sigma, s) : e, \sigma, s, pc + l_2 + l_1 + 1 \rangle && \text{(operational semantics)} \\
&= \langle c, \mathcal{A}(a_1/a_2, \sigma, s) : e, \sigma, s, pc + l_2 + l_1 + 1 \rangle && \text{(definition of } \mathcal{A} \text{)} \\
&= \langle c, \mathcal{A}(a, \sigma, s) : e, \sigma, s, pc + l_2 + l_1 + 1 \rangle
\end{aligned}$$

– **subcase 7.2.2:** $\mathcal{A}(a_2, \sigma, s) = 0.0$

$$\begin{aligned}
\langle \mathcal{CA}(a) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CA}(a_1/a_2) : c, e, \sigma, s, pc \rangle \\
&= \langle \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \text{fDIV} : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CA} \text{)} \\
&\triangleright^* \langle \mathcal{CA}(a_1) : \text{fDIV} : c, \mathcal{A}(a_2, \sigma, s) : e, \sigma, s, pc + l_2 \rangle && \text{(induction hypothesis 2)} \\
&= \langle \mathcal{CA}(a_1) : \text{fDIV} : c, 0.0 : e, \sigma, s, pc + l_2 \rangle \\
&\triangleright^* \langle \text{fDIV} : c, \mathcal{A}(a_1, \sigma, s) : 0.0 : e, \sigma, s, pc + l_2 + l_1 \rangle && \text{(induction hypothesis 1)} \\
&\triangleright \langle c, \perp : e, \hat{\sigma}, s, pc + l_2 + l_1 + 1 \rangle && \text{(operational semantics)} \\
&= \langle c, \mathcal{A}(a_1/a_2, \sigma, s) : e, \hat{\sigma}, s, pc + l_2 + l_1 + 1 \rangle && \text{(definition of } \mathcal{A} \text{)} \\
&= \langle c, \mathcal{A}(a, \sigma, s) : e, \hat{\sigma}, s, pc + l_2 + l_1 + 1 \rangle
\end{aligned}$$

where l_1 and l_2 are the number of commands in $\mathcal{CA}(a_1)$ and $\mathcal{CA}(a_2)$, respectively. Observe that, at each case, the increase in pc is exactly equal to the number of commands in $\mathcal{CA}(a)$, as desired. □

4.1.2 Boolean

Given any program stack e , non-erroneous program state σ , scope s , program counter pc ; if b does not have a erroneous subterm:

$$\forall b \in \mathbf{BExp} \cdot \langle \mathcal{CB}(b) : c, e, \sigma, s, pc \rangle \triangleright^* \langle c, \mathcal{B}(b, \sigma, s) : e, \sigma, s, pc + l \rangle$$

where l is the number of commands in $\mathcal{CB}(b)$.

Proof. We will prove this by an induction on the structure of boolean expressions.

- **case 1:** $b = \mathbf{true}$ for some $x \in \mathbf{Imm} \cap (\mathbf{Int} \cup \mathbf{Float})$

$$\begin{aligned} \langle \mathcal{CB}(b) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CB}(\mathbf{true}) : c, e, \sigma, s, pc \rangle \\ &= \langle \mathbf{PUSH\ 1} : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CB} \text{)} \\ &\triangleright \langle c, \mathcal{V}(\mathbf{1}) : e, \sigma, s, pc + 1 \rangle && \text{(operational semantics)} \\ &= \langle c, 1 : e, \sigma, s, pc + 1 \rangle \\ &= \langle c, \mathcal{B}(\mathbf{true}, \sigma, s) : e, \sigma, s, pc + 1 \rangle && \text{(definition of } \mathcal{B} \text{)} \\ &= \langle c, \mathcal{B}(b, \sigma, s) : e, \sigma, s, pc + 1 \rangle \end{aligned}$$

- **case 2:** $b = \mathbf{false}$ for some $x \in \mathbf{Var}$

$$\begin{aligned} \langle \mathcal{CB}(b) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CB}(\mathbf{false}) : c, e, \sigma, s, pc \rangle \\ &= \langle \mathbf{PUSH\ 0} : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CB} \text{)} \\ &\triangleright \langle c, \mathcal{V}(\mathbf{0}) : e, \sigma, s, pc + 1 \rangle && \text{(operational semantics)} \\ &= \langle c, 0 : e, \sigma, s, pc + 1 \rangle \\ &= \langle c, \mathcal{B}(\mathbf{false}, \sigma, s) : e, \sigma, s, pc + 1 \rangle && \text{(definition of } \mathcal{B} \text{)} \\ &= \langle c, \mathcal{B}(b, \sigma, s) : e, \sigma, s, pc + 1 \rangle \end{aligned}$$

- **case 3:** $b \in \mathbf{AExp}$

- **subcase 3.1:** $\mathcal{A}(a, \sigma, s) \neq 0$

$$\begin{aligned} \langle \mathcal{CB}(b) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CB}(a) : c, e, \sigma, s, pc \rangle \\ &= \langle \mathcal{CA}(a) : \mathbf{PUSH\ 0} : \mathbf{EQ} : \mathbf{NEG} : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CB} \text{)} \\ &\triangleright^* \langle \mathbf{PUSH\ 0} : \mathbf{EQ} : \mathbf{NEG} : c, \mathcal{A}(a, \sigma, s) : e, \sigma, s, pc + l_a \rangle && \text{(correctness of } \mathcal{CA} \text{)} \\ &\triangleright \langle \mathbf{EQ} : \mathbf{NEG} : c, 0 : \mathcal{A}(a, \sigma, s) : e, \sigma, s, pc + l_a + 1 \rangle && \text{(operational semantics)} \\ &\triangleright \langle \mathbf{NEG} : c, (0 = \mathcal{A}(a, \sigma, s)) : e, \sigma, s, pc + l_a + 2 \rangle && \text{(operational semantics)} \\ &= \langle \mathbf{NEG} : c, 0 : e, \sigma, s, pc + l_a + 2 \rangle && \text{(boolean algebra)} \\ &\triangleright \langle c, 1 : e, \sigma, s, pc + l_a + 3 \rangle && \text{(operational semantics)} \\ &= \langle c, \mathcal{B}(a, \sigma, s) : e, \sigma, s, pc + l_a + 3 \rangle && \text{(definition of } \mathcal{B} \text{)} \\ &= \langle c, \mathcal{B}(b, \sigma, s) : e, \sigma, s, pc + l_a + 3 \rangle \end{aligned}$$

– **subcase 3.2:** $\mathcal{A}(a, \sigma, s) = 0$

$$\begin{aligned}
\langle \mathcal{CB}(b) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CB}(a) : c, e, \sigma, s, pc \rangle \\
&= \langle \mathcal{CA}(a) : \text{PUSH } 0 : \text{EQ} : \text{NEG} : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CB} \text{)} \\
&\triangleright^* \langle \text{PUSH } 0 : \text{EQ} : \text{NEG} : c, \mathcal{A}(a, \sigma, s) : e, \sigma, s, pc + l_a \rangle && \text{(correctness of } \mathcal{CA} \text{)} \\
&\triangleright \langle \text{EQ} : \text{NEG} : c, 0 : \mathcal{A}(a, \sigma, s) : e, \sigma, s, pc + l_a + 1 \rangle && \text{(operational semantics)} \\
&\triangleright \langle \text{NEG} : c, (0 = \mathcal{A}(a, \sigma, s)) : e, \sigma, s, pc + l_a + 2 \rangle && \text{(operational semantics)} \\
&= \langle \text{NEG} : c, (0 = 0) : e, \sigma, s, pc + l_a + 2 \rangle \\
&= \langle \text{NEG} : c, 1 : e, \sigma, s, pc + l_a + 2 \rangle && \text{(boolean algebra)} \\
&\triangleright \langle c, 0 : e, \sigma, s, pc + l_a + 3 \rangle && \text{(operational semantics)} \\
&= \langle c, \mathcal{B}(a, \sigma, s) : e, \sigma, s, pc + l_a + 3 \rangle && \text{(definition of } \mathcal{B} \text{)} \\
&= \langle c, \mathcal{B}(b, \sigma, s) : e, \sigma, s, pc + l_a + 3 \rangle
\end{aligned}$$

• **case 4:** $b = (a_1 = a_2)$ for some $a_1, a_2 \in \mathbf{AExp}$

– **subcase 4.1:** $\mathcal{A}(a_1, \sigma, s) = \mathcal{A}(a_2, \sigma, s)$

$$\begin{aligned}
\langle \mathcal{CB}(b) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CB}(a_1 = a_2) : c, e, \sigma, s, pc \rangle \\
&= \langle \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \text{EQ} : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CB} \text{)} \\
&\triangleright^* \langle \mathcal{CA}(a_1) : \text{EQ} : c, \mathcal{A}(a_2, \sigma, s) : e, \sigma, s, pc + l_{a_1} \rangle && \text{(correctness of } \mathcal{CA} \text{)} \\
&\triangleright^* \langle \text{EQ} : c, \mathcal{A}(a_1, \sigma, s) : \mathcal{A}(a_2, \sigma, s) : e, \sigma, s, pc + l_{a_1} + l_{a_2} \rangle && \text{(correctness of } \mathcal{CA} \text{)} \\
&\triangleright \langle c, (\mathcal{A}(a_1, \sigma, s) = \mathcal{A}(a_2, \sigma, s)) : e, \sigma, s, pc + l_{a_1} + l_{a_2} + 1 \rangle && \text{(operational semantics)} \\
&= \langle c, 1 : e, \sigma, s, pc + l_{a_1} + l_{a_2} + 1 \rangle \\
&= \langle c, \mathcal{B}(a_1 = a_2, \sigma, s) : e, \sigma, s, pc + l_{a_1} + l_{a_2} + 1 \rangle && \text{(definition of } \mathcal{B} \text{)} \\
&= \langle c, \mathcal{B}(b, \sigma, s) : e, \sigma, s, pc + l_{a_1} + l_{a_2} + 1 \rangle
\end{aligned}$$

– **subcase 4.1:** $\mathcal{A}(a_1, \sigma, s) \neq \mathcal{A}(a_2, \sigma, s)$

$$\begin{aligned}
\langle \mathcal{CB}(b) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CB}(a_1 = a_2) : c, e, \sigma, s, pc \rangle \\
&= \langle \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \text{EQ} : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CB} \text{)} \\
&\triangleright^* \langle \mathcal{CA}(a_1) : \text{EQ} : c, \mathcal{A}(a_2, \sigma, s) : e, \sigma, s, pc + l_{a_1} \rangle && \text{(correctness of } \mathcal{CA} \text{)} \\
&\triangleright^* \langle \text{EQ} : c, \mathcal{A}(a_1, \sigma, s) : \mathcal{A}(a_2, \sigma, s) : e, \sigma, s, pc + l_{a_1} + l_{a_2} \rangle && \text{(correctness of } \mathcal{CA} \text{)} \\
&\triangleright \langle c, (\mathcal{A}(a_1, \sigma, s) = \mathcal{A}(a_2, \sigma, s)) : e, \sigma, s, pc + l_{a_1} + l_{a_2} + 1 \rangle && \text{(operational semantics)} \\
&= \langle c, 0 : e, \sigma, s, pc + l_{a_1} + l_{a_2} + 1 \rangle \\
&= \langle c, \mathcal{B}(a_1 = a_2, \sigma, s) : e, \sigma, s, pc + l_{a_1} + l_{a_2} + 1 \rangle && \text{(definition of } \mathcal{B} \text{)} \\
&= \langle c, \mathcal{B}(b, \sigma, s) : e, \sigma, s, pc + l_{a_1} + l_{a_2} + 1 \rangle
\end{aligned}$$

• **case 5:** $b = (\text{not } b')$ for some $b' \in \mathbf{BExp}$

Assume for an induction on the correctness of the compilation of b' .

$$\begin{aligned}
\langle \mathcal{CB}(b) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CB}(\text{not } b') : c, e, \sigma, s, pc \rangle \\
&= \langle \mathcal{CB}(b') : \text{NEG} : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CB} \text{)} \\
&\triangleright^* \langle \text{NEG} : c, \mathcal{B}(b', \sigma, s) : e, \sigma, s, pc + l_{b'} \rangle && \text{(induction hypothesis)} \\
&\triangleright \langle c, \neg \mathcal{B}(b', \sigma, s) : e, \sigma, s, pc + l_{b'} + 1 \rangle && \text{(operational semantics)} \\
&= \langle c, \mathcal{B}(\text{not } b', \sigma, s) : e, \sigma, s, pc + l_{b'} + 1 \rangle && \text{(definition of } \mathcal{B} \text{)} \\
&= \langle c, \mathcal{B}(b, \sigma, s) : e, \sigma, s, pc + l_{b'} + 1 \rangle
\end{aligned}$$

- **case 6:** $b = (b_1 \text{ and } b_2)$ for some $b_1, b_2 \in \mathbf{BExp}$

Assume for an induction on the correctness of the compilations of b_1 and b_2 .

$$\begin{aligned}
\langle \mathcal{CB}(b) : c, e, \sigma, s, pc \rangle &= \langle \mathcal{CB}(b_1 \text{ and } b_2) : c, e, \sigma, s, pc \rangle \\
&= \langle \mathcal{CB}(b_2) : \mathcal{CB}(b_1) : \text{AND} : c, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{CB} \text{)} \\
&\triangleright^* \langle \mathcal{CB}(b_1) : \text{AND} : c, \mathcal{B}(b_2, \sigma, s) : e, \sigma, s, pc + l_{b_2} \rangle && \text{(induction hypothesis 2)} \\
&\triangleright^* \langle \text{AND} : c, \mathcal{B}(b_1, \sigma, s) : \mathcal{B}(b_2, \sigma, s) : e, \sigma, s, pc + l_{b_2} + l_{b_1} \rangle && \text{(induction hypothesis 2)} \\
&\triangleright \langle c, (\mathcal{B}(b_1, \sigma, s) \wedge \mathcal{B}(b_2, \sigma, s)) : e, \sigma, s, pc + l_{b_2} + l_{b_1} + 1 \rangle && \text{(operational semantics)} \\
&= \langle c, \mathcal{B}(b_1 \text{ and } b_2, \sigma, s) : e, \sigma, s, pc + l_{b_2} + l_{b_1} + 1 \rangle && \text{(definition of } \mathcal{B} \text{)} \\
&= \langle c, \mathcal{B}(b, \sigma, s) : e, \sigma, s, pc + l_{b_2} + l_{b_1} + 1 \rangle
\end{aligned}$$

where $l_{a'}$ denotes the number of commands in $\mathcal{CA}(a')$ for all $a' \in \mathbf{AExp}$ and $l_{b'}$ denotes the number of commands in $\mathcal{CB}(b')$ for all $b' \in \mathbf{BExp}$.

Observe that in each case the increase in pc is exactly equal to the number of commands in $\mathcal{CB}(b)$, as desired. $\square$

4.2 Correctness of the statements

Now, as the correctness of arithmetic and boolean expressions is proven, we may restate and prove the correctness of our compiler:

$$\langle P, \underline{\sigma}, s \rangle \rightarrow^* \langle \underline{\sigma}_f, s_f \rangle \implies \langle \mathcal{C}(P), e, \underline{\sigma}, s, pc \rangle \triangleright^* \langle \underline{\sigma}_f, s_f \rangle$$

where P is any **SelVeri** program.

Observe that the implication above mentions the correctness of only terminating programs. However, we also claim that the compilation of non-terminating **SelVeri** programs are also correct, and we will present our arguments in the **case 6** of the following proof.

Proof. If $\underline{\sigma}$ is an erroneous program state, then by section 2.2.4 of this document and the operational semantic rule of high-level **SelVeri** language [err_{os}], the above implication is obviously true⁷. Therefore, from now on, we assume that $\underline{\sigma}$ represents a non-erroneous program state and will use σ to denote it.

We will prove the correctness of $\mathcal{C}$ via a structural induction on **SelVeri** statements, knowing that P itself is actually a statement⁸.

- **case 1:** $P = (x : t)$

– **subcase 1.1:** $t \in \mathbf{Type} \wedge s(x) = \mathbf{Void}$

$$\begin{aligned}
\langle P, \sigma, s \rangle &= \langle x : t, \sigma, s \rangle \\
&\rightarrow \langle \sigma[x \mapsto \mathbf{0}], s[x \mapsto t] \rangle \quad [\text{decl}_{\text{os}}]
\end{aligned}$$

and

$$\begin{aligned}
\langle \mathcal{C}(P), e, \sigma, s, pc \rangle &= \langle \mathcal{C}(x : t), e, \sigma, s, pc \rangle \\
&= \langle \text{DECL } t, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{C} \text{)} \\
&= \langle e, e, \sigma[x \mapsto \mathbf{0}], s[x \mapsto t], pc + 1 \rangle && \text{(operational semantics)}
\end{aligned}$$

⁷Furthermore, $\sigma = \sigma_f$ and $s = s_f$, in this case.

⁸It is mostly likely a long chain of compositional statements.

– **subcase 1.2:** $t \notin \mathbf{Type} \vee s(x) \neq \mathbf{Void}$

$$\begin{aligned} \langle P, \sigma, s \rangle &= \langle x : t, \sigma, s \rangle \\ &\rightarrow \hat{\sigma} \quad [\text{decl}_{\text{os}}^\perp] \end{aligned}$$

and

$$\begin{aligned} \langle \mathcal{C}(P), e, \sigma, s, pc \rangle &= \langle \mathcal{C}(x : t), e, \sigma, s, pc \rangle \\ &= \langle \mathbf{DECL} \ t, e, \sigma, s, pc \rangle \quad (\text{definition of } \mathcal{C}) \\ &= \langle \epsilon, e, \hat{\sigma}, s, pc + 1 \rangle \quad (\text{operational semantics}) \end{aligned}$$

• **case 2:** $P = (x := a)$

Assume $\mathcal{A}(a, \sigma, s) \neq \perp \wedge \mathcal{A}(a, \sigma, s) \in \text{ran} \mathcal{L}_{s(x)}$, then:

– **subcase 2.1:** $s(x) = \mathbf{Int}$

$$\begin{aligned} \langle P, \sigma, s \rangle &= \langle x := a, \sigma, s \rangle \\ &\quad \sigma[x \mapsto \mathcal{A}(a, \sigma, s)] \quad [\text{ass}_{\text{os}}] \end{aligned}$$

and

$$\begin{aligned} \langle \mathcal{C}(P), e, \sigma, s, pc \rangle &= \langle \mathcal{C}(x := a), e, \sigma, s, pc \rangle \\ &= \langle \mathcal{CA}(a) : \mathbf{iSTORE} \ x, e, \sigma, s, pc \rangle \quad (\text{definition of } \mathcal{C}) \\ &\triangleright^* \langle \mathbf{iSTORE} \ x, \mathcal{A}(a, \sigma, s) : e, \sigma, s, pc + l_a \rangle \quad (\text{correctness of } \mathcal{CA}) \\ &= \langle \epsilon, e, \sigma[x \mapsto \mathcal{A}(a, \sigma, s)], s, pc + l_a + 1 \rangle \quad (\text{operational semantics}) \end{aligned}$$

– **subcase 2.2:** $s(x) = \mathbf{Float}$

$$\begin{aligned} \langle P, \sigma, s \rangle &= \langle x := a, \sigma, s \rangle \\ &\quad \sigma[x \mapsto \mathcal{A}(a, \sigma, s)] \quad [\text{ass}_{\text{os}}] \end{aligned}$$

and

$$\begin{aligned} \langle \mathcal{C}(P), e, \sigma, s, pc \rangle &= \langle \mathcal{C}(x := a), e, \sigma, s, pc \rangle \\ &= \langle \mathcal{CA}(a) : \mathbf{fSTORE} \ x, e, \sigma, s, pc \rangle \quad (\text{definition of } \mathcal{C}) \\ &\triangleright^* \langle \mathbf{fSTORE} \ x, \mathcal{A}(a, \sigma, s) : e, \sigma, s, pc + l_a \rangle \quad (\text{correctness of } \mathcal{CA}) \\ &= \langle \epsilon, e, \sigma[x \mapsto \mathcal{A}(a, \sigma, s)], s, pc + l_a + 1 \rangle \quad (\text{operational semantics}) \end{aligned}$$

• **case 3:** $P = \text{pass}$

$$\begin{aligned} \langle P, \sigma, s \rangle &= \langle \text{pass}, \sigma, s \rangle \\ &\rightarrow \langle \sigma, s \rangle \quad [\text{pass}_{\text{os}}] \end{aligned}$$

and

$$\begin{aligned} \langle \mathcal{C}(P), e, \sigma, s, pc \rangle &= \langle \mathcal{C}(\text{pass}), e, \sigma, s, pc \rangle \\ &= \langle \mathbf{NOOP}, e, \sigma, s, pc \rangle \quad (\text{definition of } \mathcal{C}) \\ &= \langle \epsilon, e, \sigma, s, pc + 1 \rangle \quad (\text{operational semantics}) \end{aligned}$$

- **case 4:** $P = (\text{if } b \text{ then } S \text{ fi})$

- **subcase 4.1:** $\mathcal{B}(b, \sigma, s) = 1$

Assume

$$\begin{aligned}
\langle P, \sigma, s \rangle &= \langle \text{if } b \text{ then } S \text{ fi}, \sigma, s \rangle \\
&= \langle \text{if } b \text{ then } S \text{ else pass fi}, \sigma, s \rangle \quad (\text{semantical equivalence}) \\
&\rightarrow \langle S, \sigma, s \rangle \quad ([\text{if}_{\text{os}}^1]) \\
&\rightarrow^* \langle \underline{\sigma}', s' \rangle
\end{aligned}$$

and assume for an induction hypothesis, that is, the compilation of S is correct, then

$$\begin{aligned}
\langle \mathcal{C}(P), e, \sigma, s, pc \rangle &= \langle \mathcal{C}(\text{if } b \text{ then } S \text{ fi}), e, \sigma, s, pc \rangle \\
&= \langle \mathcal{CB}(b) : \text{JZ } pc_{\text{NOOP}} : \mathcal{C}(S) : \text{NOOP}, e, \sigma, s, pc \rangle \quad (\text{definition of } \mathcal{C}) \\
&\triangleright^* \langle \text{JZ } pc_{\text{NOOP}} : \mathcal{C}(S) : \text{NOOP}, \mathcal{B}(b, \sigma, s) : e, \sigma, s, pc + l_b \rangle \quad (\text{correctness of } \mathcal{CB}) \\
&= \langle \text{JZ } pc_{\text{NOOP}} : \mathcal{C}(S) : \text{NOOP}, 1 : e, \sigma, s, pc + l_b \rangle \\
&\triangleright \langle \mathcal{C}(S) : \text{NOOP}, e, \sigma, s, pc + l_b + 1 \rangle \quad (\text{operational semantics}) \\
&\triangleright^* \langle \text{NOOP}, e', \underline{\sigma}', s', pc + l_b + l_S + 1 \rangle \quad (\text{induction hypothesis}) \\
&\triangleright \langle \epsilon, e', \underline{\sigma}', s', pc + l_b + l_S + 2 \rangle \quad (\text{operational semantics})
\end{aligned}$$

- **subcase 4.2:** $\mathcal{B}(b, \sigma, s) = 0$

Assume

$$\begin{aligned}
\langle P, \sigma, s \rangle &= \langle \text{if } b \text{ then } S \text{ fi}, \sigma, s \rangle \\
&= \langle \text{if } b \text{ then } S \text{ else pass fi}, \sigma, s \rangle \quad (\text{semantical equivalence}) \\
&\rightarrow \langle \text{pass}, \sigma, s \rangle \quad [\text{if}_{\text{os}}^0] \\
&\rightarrow \langle \sigma, s \rangle \quad [\text{pass}_{\text{os}}]
\end{aligned}$$

then,

$$\begin{aligned}
\langle \mathcal{C}(P), e, \sigma, s, pc \rangle &= \langle \mathcal{C}(\text{if } b \text{ then } S \text{ fi}), e, \sigma, s, pc \rangle \\
&= \langle \mathcal{CB}(b) : \text{JZ } pc_{\text{NOOP}} : \mathcal{C}(S) : \text{NOOP}, e, \sigma, s, pc \rangle \quad (\text{definition of } \mathcal{C}) \\
&\triangleright^* \langle \text{JZ } pc_{\text{NOOP}} : \mathcal{C}(S) : \text{NOOP}, \mathcal{B}(b, \sigma, s) : e, \sigma, s, pc + l_b \rangle \quad (\text{correctness of } \mathcal{CB}) \\
&= \langle \text{JZ } pc_{\text{NOOP}} : \mathcal{C}(S) : \text{NOOP}, 0 : e, \sigma, s, pc + l_b \rangle \\
&\triangleright \langle \text{NOOP}, e, \sigma, s, pc_{\text{NOOP}} \rangle \quad (\text{operational semantics}) \\
&\triangleright \langle \epsilon, e, \sigma, s, pc_{\text{NOOP}} + 1 \rangle \quad (\text{operational semantics})
\end{aligned}$$

- **case 5:** $P = (\text{if } b \text{ then } S_1 \text{ else } S_2 \text{ fi})$

- **subcase 5.1:** $\mathcal{B}(b, \sigma, s) = 1$

Assume

$$\begin{aligned}
\langle P, \sigma, s \rangle &= \langle \text{if } b \text{ then } S_1 \text{ else } S_2 \text{ fi}, \sigma, s \rangle \\
&\rightarrow \langle S_1, \sigma, s \rangle \quad [\text{if}_{\text{os}}^1] \\
&\rightarrow^* \langle \underline{\sigma}', s' \rangle
\end{aligned}$$

and assume for an induction hypothesis, that is, the compilation of S_1 is correct, then

$$\begin{aligned}
\langle \mathcal{C}(P), e, \sigma, s, pc \rangle &= \langle \mathcal{C}(\text{if } b \text{ then } S_1 \text{ else } S_2 \text{ fi}), e, \sigma, s, pc \rangle \\
&= \langle \mathcal{CB}(b) : \text{JZ } pc_{S_2} : \mathcal{C}(S_1) : \text{GOTO } pc_{\text{NOOP}} : \mathcal{C}(S_2) : \text{NOOP}, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{C} \text{)} \\
&\triangleright^* \langle \text{JZ } pc_{S_2} : \mathcal{C}(S_1) : \text{GOTO } pc_{\text{NOOP}} : \mathcal{C}(S_2) : \text{NOOP}, \mathcal{B}(b, \sigma, s) : e, \sigma, s, pc + l_b \rangle && \text{(correctness of } \mathcal{CB} \text{)} \\
&= \langle \text{JZ } pc_{S_2} : \mathcal{C}(S_1) : \text{GOTO } pc_{\text{NOOP}} : \mathcal{C}(S_2) : \text{NOOP}, 1 : e, \sigma, s, pc + l_b \rangle \\
&\triangleright \langle \mathcal{C}(S_1) : \text{GOTO } pc_{\text{NOOP}} : \mathcal{C}(S_2) : \text{NOOP}, e, \sigma, s, pc + l_b + 1 \rangle && \text{(operational semantics)} \\
&\triangleright^* \langle \text{GOTO } pc_{\text{NOOP}} : \mathcal{C}(S_2) : \text{NOOP}, e', \underline{\sigma}', s', pc + l_b + l_{S_1} + 1 \rangle && \text{(induction hypothesis)} \\
&\triangleright \langle \text{NOOP}, e', \underline{\sigma}', s', pc_{\text{NOOP}} \rangle && \text{(operational semantics)} \\
&\triangleright \langle \epsilon, e', \underline{\sigma}', s', pc_{\text{NOOP}} + 1 \rangle && \text{(operational semantics)}
\end{aligned}$$

– **subcase 5.2:** $\mathcal{B}(b, \sigma, s) = 0$

Assume

$$\begin{aligned}
\langle P, \sigma, s \rangle &= \langle \text{if } b \text{ then } S_1 \text{ else } S_2 \text{ fi}, \sigma, s \rangle \\
&\rightarrow \langle S_2, \sigma, s \rangle && [\text{if}_{\text{os}}^0] \\
&\rightarrow^* \langle \underline{\sigma}', s' \rangle
\end{aligned}$$

and assume for an induction hypothesis, that is, the compilation of S_2 is correct, then

$$\begin{aligned}
\langle \mathcal{C}(P), e, \sigma, s, pc \rangle &= \langle \mathcal{C}(\text{if } b \text{ then } S_1 \text{ else } S_2 \text{ fi}), e, \sigma, s, pc \rangle \\
&= \langle \mathcal{CB}(b) : \text{JZ } pc_{S_2} : \mathcal{C}(S_1) : \text{GOTO } pc_{\text{NOOP}} : \mathcal{C}(S_2) : \text{NOOP}, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{C} \text{)} \\
&\triangleright^* \langle \text{JZ } pc_{S_2} : \mathcal{C}(S_1) : \text{GOTO } pc_{\text{NOOP}} : \mathcal{C}(S_2) : \text{NOOP}, \mathcal{B}(b, \sigma, s) : e, \sigma, s, pc + l_b \rangle && \text{(correctness of } \mathcal{CB} \text{)} \\
&= \langle \text{JZ } pc_{S_2} : \mathcal{C}(S_1) : \text{GOTO } pc_{\text{NOOP}} : \mathcal{C}(S_2) : \text{NOOP}, 0 : e, \sigma, s, pc + l_b \rangle \\
&\triangleright \langle \mathcal{C}(S_2) : \text{NOOP}, e, \sigma, s, pc_{S_2} \rangle && \text{(operational semantics)} \\
&\triangleright^* \langle \text{NOOP}, e', \underline{\sigma}', s', pc_{S_2} + 1 \rangle && \text{(induction hypothesis)} \\
&\triangleright \langle \epsilon, e', \underline{\sigma}', s', pc_{S_2} + 2 \rangle && \text{(operational semantics)}
\end{aligned}$$

• **case 6:** $P = (\text{while } b \text{ do } S \text{ od})$

– **subcase 6.1:** $\mathcal{B}(b, \sigma, s) = 0$

Assume

$$\begin{aligned}
\langle P, \sigma, s \rangle &= \langle \text{while } b \text{ do } S \text{ od}, \sigma, s \rangle \\
&\rightarrow \langle \sigma, s \rangle && [\text{while}_{\text{os}}^0]
\end{aligned}$$

then,

$$\begin{aligned}
\langle \mathcal{C}(P), e, \sigma, s, pc \rangle &= \langle \mathcal{C}(\text{while } b \text{ do } S \text{ od}), e, \sigma, s, pc \rangle \\
&= \langle \mathcal{CB}(b) : \text{JZ } pc_{\text{NOOP}} : \mathcal{C}(S) : \text{GOTO } pc_b : \text{NOOP}, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{C} \text{)} \\
&\triangleright^* \langle \text{JZ } pc_{\text{NOOP}} : \mathcal{C}(S) : \text{GOTO } pc_b : \text{NOOP}, \mathcal{B}(b, \sigma, s) : e, \sigma, s, pc + l_b \rangle && \text{(correctness of } \mathcal{CB} \text{)} \\
&= \langle \text{JZ } pc_{\text{NOOP}} : \mathcal{C}(S) : \text{GOTO } pc_b : \text{NOOP}, 0 : e, \sigma, s, pc + l_b \rangle \\
&\triangleright \langle \text{NOOP}, e, \sigma, s, pc_{\text{NOOP}} \rangle && \text{(operational semantics)} \\
&\triangleright \langle \epsilon, e, \sigma, s, pc_{\text{NOOP}} + 1 \rangle && \text{(operational semantics)}
\end{aligned}$$

– **subcase 6.2:** $\mathcal{B}(b, \sigma, s) = 1$

$$\begin{aligned}
\langle P, \sigma, s \rangle &= \langle \text{while } b \text{ do } S \text{ od}, \sigma, s \rangle \\
&\rightarrow \langle S; \text{while } b \text{ do } S \text{ od}, \sigma, s \rangle && [\text{while}_{\text{os}}^1] \\
&\rightarrow \langle \text{while } b \text{ do } S \text{ od}, \underline{\sigma}', s' \rangle
\end{aligned}$$

and assume for an induction hypothesis, that is, the compilation of S is correct, then

$$\begin{aligned}
\langle \mathcal{C}(P), e, \sigma, s, pc \rangle &= \langle \mathcal{C}(\text{while } b \text{ do } S \text{ od}), e, \sigma, s, pc \rangle \\
&= \langle \mathcal{CB}(b) : \text{JZ } pc_{\text{NOOP}} : \mathcal{C}(S) : \text{GOTO } pc_b : \text{NOOP}, e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{C} \text{)} \\
&\triangleright^* \langle \text{JZ } pc_{\text{NOOP}} : \mathcal{C}(S) : \text{GOTO } pc_b : \text{NOOP}, \mathcal{B}(b, \sigma, s) : e, \sigma, s, pc + l_b \rangle && \text{(correctness of } \mathcal{CB} \text{)} \\
&= \langle \text{JZ } pc_{\text{NOOP}} : \mathcal{C}(S) : \text{GOTO } pc_b : \text{NOOP}, 1 : e, \sigma, s, pc + l_b \rangle \\
&\triangleright \langle \mathcal{C}(S) : \text{GOTO } pc_b : \text{NOOP}, e, \sigma, s, pc + l_b + 1 \rangle && \text{(operational semantics)} \\
&\triangleright^* \langle \text{GOTO } pc_b : \text{NOOP}, e', \underline{\sigma}', s', pc + l_b + l_S + 1 \rangle && \text{(induction hypothesis)} \\
&\triangleright \langle \mathcal{CB}(b) : \text{JZ } pc_{\text{NOOP}} : \mathcal{C}(S) : \text{GOTO } pc_b : \text{NOOP}, e', \underline{\sigma}', s', pc_b \rangle && \text{(operational semantics)} \\
&= \langle \mathcal{CB}(b) : \text{JZ } pc_{\text{NOOP}} : \mathcal{C}(S) : \text{GOTO } pc_b : \text{NOOP}, e', \underline{\sigma}', s', pc \rangle \\
&= \langle \mathcal{C}(\text{while } b \text{ do } S \text{ od}), e', \underline{\sigma}', s', pc \rangle
\end{aligned}$$

However, the proofs above are not enough to establish the correctness of the compiled **while**-statements, because they only demonstrate one step of iteration. Nonetheless, with a simple induction, we can generalize this argument to claim that at any step of iteration, the compilation is correct, as the proofs above show that at any given program state and scope, the compilation of one iteration is correct. To put it more explicitly, the cases of 0 iterations (**subcase 6.1**) and 1 iteration (**subcase 6.2**) are proven correct. Assuming that i iterations are compiled correctly, **subcase 6.2** also prove that $i+1$ iterations are compiled correctly. Ultimately, one can certainly say that any terminating **SelVeri** loop is compiled correctly.

Until so far, we have omitted non-terminating programs in our correctness proof. But, the only possible source of non-termination in **SelVeri** is a **while**-loop where the semantical value of the boolean guard $b = 1$ is a loop invariant. Although we cannot talk about a terminating program state in such a case; at any execution step, we know that the execution thus far has been correct and will continue to be correct, therefore one can also claim that the compilation of non-terminating **SelVeri** programs are also correct.

- **case 7:** $P = S_1; S_2$ for some statements S_1, S_2

Assume

$$\begin{aligned}
\langle P, \sigma, s \rangle &= \langle S_1; S_2, \sigma, s \rangle \\
&\rightarrow^* \langle S_2, \underline{\sigma}', s' \rangle && ([\text{comp}_{\text{os}}^1], [\text{comp}_{\text{os}}^2]) \\
&\rightarrow^* \langle \underline{\sigma}'', s'' \rangle && ([\text{comp}_{\text{os}}^1], [\text{comp}_{\text{os}}^2])
\end{aligned}$$

and, assume for an induction, that is the compilation of S_1 and S_2 are correct, then

$$\begin{aligned}
\langle \mathcal{C}(P), e, \sigma, s, pc \rangle &= \langle \mathcal{C}(S_1; S_2), e, \sigma, s, pc \rangle \\
&= \langle \mathcal{C}(S_1) : \mathcal{C}(S_2), e, \sigma, s, pc \rangle && \text{(definition of } \mathcal{C} \text{)} \\
&= \langle \mathcal{C}(S_2), e', \underline{\sigma}', s', pc + l_{S_1} \rangle && \text{(induction hypothesis 1)} \\
&= \langle \epsilon, e'', \underline{\sigma}'', s'', pc + l_{S_1} + l_{S_2} \rangle && \text{(induction hypothesis 2)}
\end{aligned}$$

Observe that at each case, the resulting program state and scope tuple is the equal for **SelVeri** and **SelVerIR**, therefore concluding the proof. □

Cautious readers may notice that some (sub)cases including erroneous program states (either as input or output) is omitted in our correctness proofs. One of the following reasons is the justification for this omittance:

1. Such a case cannot be encountered in the range of neither $\mathcal{CA}$ nor $\mathcal{CB}$ nor $\mathcal{C}$.
2. Such an erroneous case can be detected during compile-time, therefore no **SelVerIR** code inheriting the error can be generated.

No matter the reason, as no statically-correct **SelVeri** program includes any of the omitted cases, the correctness proofs above are sufficient for establishing the correctness of our implementation.